
Tarea de Minería de Datos y Modelización predictiva I

 nticmaster



U N I V E R S I D A D
COMPLUTENSE
M A D R I D

Carlos San Román Cazorla

Universidad Complutense de Madrid

Máster de IA, Big Data y Data Science

Mayo 2026

v. 1.0.0

Índice general

1. Enunciado	3
2. Depuración	5
2.1. Introducción al problema y las variables implicadas	5
2.1.1. Introducción y contexto	5
2.1.2. Identificación geográfica	5
2.1.3. Variables electorales	6
2.1.4. Variables demográficas	6
2.1.5. Variables de desempleo	7
2.1.6. Variables del tejido empresarial	7
2.1.7. Variables territoriales y de evolución demográfica	7
2.1.8. Consideraciones sobre la calidad del dato	8
2.2. Importación de datos y asignación correcta de los tipos de variables	8
2.3. Análisis descriptivo del conjunto de datos	10
2.3.1. Variables numéricas	10
2.3.2. Variables categóricas	11
2.4. Corrección de los errores detectados	11
2.5. Análisis de valores atípicos	12
2.6. Análisis de valores perdidos	13
3. Regresión Lineal	15
4. Regresión Logística	20
4.1. Determinación del punto de corte óptimo	24
4.2. Importancia de las variables en el modelo	26
5. Conclusión	27

1 Enunciado

El conjunto de datos `DatosEleccionesEspaña.xlsx` contiene información demográfica sobre los distintos municipios de España junto con los resultados que se obtuvieron en las últimas elecciones. Existen 7 posibles variables objetivo:

- **AbstentionPtge**: Porcentaje de abstención
- **Izda Pct**: Porcentaje de votos a partidos de izquierda (PSOE y Podemos)
- **Dcha Pct**: Porcentaje de votos a partidos de derecha (PP y Ciudadanos)
- **Otros Pct**: Porcentaje de votos a partidos distintos de PP, Ciudadanos, PSOE y Podemos
- **AbstentionAlta**: Variable dicotómica que toma el valor 1 si el porcentaje de abstención es superior al 30 % y, 0, en otro caso.
- **Izquierda**: Variable dicotómica que toma el valor 1 si la suma de los votos de izquierdas es superior a la de derechas y otros y, 0, en otro caso.
- **Derecha**: Variable dicotómica que toma el valor 1 si la suma de los votos de derecha es superior a la de izquierda y otros y, 0, en otro caso.

El objetivo de esta práctica es obtener dos modelos de regresión (lineal y logística) seleccionando, de entre las 7 variables anteriores, una variable objetivo continua y otra variable objetivo binaria. El resto de las **variables objetivo que no han sido seleccionadas se eliminan del conjunto de datos**, no se utilizan como variables explicativas. La variable objetivo continua se utiliza para la construcción del modelo de regresión lineal y la variable objetivo binaria se utiliza para el modelo de regresión logística.

Antes de desarrollar los modelos de regresión, es necesario llevar a cabo un proceso de depuración de los datos. Por tanto, los pasos a seguir para la realización de la práctica son:

1. **Introducción al objetivo del problema y las variables implicadas.** Se debe explicar cuál es el objetivo del problema que se va a analizar según las variables objetivo seleccionadas. Además, explicar brevemente como son las variables implicadas en el estudio de forma general, no es necesario describir cada una de ellas.
2. **Importación** del conjunto de datos y asignación correcta de los tipos de variables.
3. **Análisis descriptivo del conjunto de datos para conocer los datos y detectar errores.** Número de observaciones, número y naturaleza de variables, datos erróneos etc.
4. **Corrección** de los errores detectados.

5. **Análisis de valores atípicos** utilizando distintas metodologías para la detección y su posterior análisis. Decisiones.
6. **Análisis de valores perdidos** por observación y por variable. Distintas decisiones a tomar: Imputaciones, eliminación, recategorización.
7. **Construcción del modelo de regresión lineal (NO hacer el modelo manual. NO utilizar transformaciones. NO utilizar interacciones)** a partir de los siguientes apartados:
 - Mediante los métodos de selección clásica de variables.
 - Selección del modelo ganador de entre todos los modelos construidos.
 - Interpretación de los valores de los coeficientes de dos variables incluidas en el modelo ganador, una categórica y otra continua.
 - Justificar por qué es el modelo ganador y medir la calidad del mismo. Número de parámetros, robustez, significación de los parámetros, importancia de las variables, etc.
8. **Construcción del modelo de regresión logística (NO hacer el modelo manual. NO utilizar transformaciones. NO utilizar interacciones).** En este apartado deben explicarse y justificarse todos los pasos a realizar para responder a los apartados solicitados en la tarea. Aunque los pasos y códigos a utilizar son muy similares en regresión lineal y logística, debe indicarse lo que es igual, haciendo referencia al apartado correspondiente en regresión lineal. Los códigos que son distintos en regresión lineal y logística, aunque muy similares, se puede destacar solo la parte del código que es diferente respecto al código detallado anteriormente en regresión lineal. El modelo de regresión logística se construye a partir de los siguientes apartados:
 - Mediante los métodos de selección clásica de variables.
 - Selección del modelo ganador de entre todos los modelos construidos. Determinar el punto de corte óptimo.
 - Interpretación de los valores de los coeficientes de dos variables incluidas en el modelo ganador, una categórica y otra continua.
 - Justificar por qué es el modelo ganador y medir la calidad del mismo. Número de parámetros, robustez, significación de los parámetros, importancia de las variables, etc.

Se entregará un informe en PDF (máximo 25 páginas, la portada y el índice no están incluidas, cualquier página adicional no se tendrá en cuenta) en el que se explicarán detalladamente los pasos seguidos incluyendo los códigos y salidas más relevantes. Imprescindible mostrar el summary del modelo ganador. Es muy importante comentar y justificar razonadamente las decisiones que se toman en cada uno de los apartados. En el documento Guía Elaboración Tarea.docx se muestra una pequeña guía sobre como elaborar el informe correspondiente a la tarea.

La puntuación de la tarea está dividida en tres partes:

- Depuración de datos (3,3 puntos). Todos los apartados de esta parte tienen la misma puntuación.
- Regresión Lineal (3,3 puntos). Todos los apartados de esta parte tienen la misma puntuación.
- Regresión Logística (3,4 puntos). Todos los apartados de esta parte tienen la misma puntuación.

2 Depuración

2.1. Introducción al problema y las variables implicadas

2.1.1. Introducción y contexto

El presente capítulo ofrece una descripción detallada del conjunto de datos empleado en este trabajo, encontrado en el archivo `DatosEleccionesEspaña.xlsx`. Dicho dataset recopila información socioeconómica, demográfica y electoral referida a los municipios españoles, con datos correspondientes probablemente al año 2016 por los partidos representados. La unidad de análisis es el municipio, que constituye la entidad administrativa básica del territorio español.

El dataset cuenta con un total de **41 variables** (columnas) y abarca municipios de todas las comunidades autónomas de España, cubriendo las 52 provincias del territorio nacional. Su riqueza multidimensional lo convierte en un instrumento adecuado para el análisis de los factores que determinan el comportamiento electoral a nivel local, así como para estudiar las desigualdades territoriales en términos de estructura socioeconómica.

A continuación se presenta una descripción sistemática de alguna de las variables que componen el dataset (no describiremos todas debido a las restricciones en el espacio de la memoria), agrupadas por categorías temáticas para facilitar su comprensión.

2.1.2. Identificación geográfica

Variable	Descripción
Name	Nombre del municipio. Es el identificador textual de la unidad de análisis. España cuenta con más de 8 000 municipios.
CodigoProvincia	Código numérico de la provincia a la que pertenece el municipio. Coincide con los dos primeros dígitos del código postal español y toma 52 valores distintos, correspondientes a las 50 provincias peninsulares e insulares más las dos ciudades autónomas de Ceuta y Melilla.
CCAA	Comunidad autónoma a la que pertenece el municipio. España se organiza territorialmente en 17 comunidades autónomas y 2 ciudades autónomas. Permite agregar los resultados a nivel regional y controlar por efectos de comunidad autónoma en los análisis estadísticos.

2.1.3. Variables electorales

Variable	Descripción
TotalCensus	Población en edad de votar en el municipio en 2016 (número de personas con 18 años o más inscritas en el censo electoral). Es el denominador de referencia para calcular tasas de participación y porcentajes de voto.
AbstentionPtge	Porcentaje de abstención sobre el censo electoral. Valores elevados pueden indicar desafección política, dificultades de acceso a las urnas o estructuras socioeconómicas particulares.
AbstencionAlta	Variable dicotómica derivada de <i>AbstentionPtge</i> . Toma el valor 1 si el porcentaje de abstención supera el 30 % y el valor 0 en caso contrario.
Izda_Pct / Dcha_Pct	Porcentaje de votos obtenidos por los partidos de izquierda (derecha) sobre el total de votos válidos emitidos. Refleja la orientación política de izquierdas (derechas) del electorado municipal.
Otros_Pct	Porcentaje de votos obtenidos por partidos distintos del PP, Ciudadanos, PSOE y Podemos.
Izquierda/ Derecha	Variable dicotómica que toma el valor 1 si la suma de los votos de izquierdas (o derechas) (<i>Izda_Pct</i>) supera a la de derechas (izquierdas) y otros partidos conjuntamente, y el valor 0 en caso contrario. Permite clasificar cada municipio según si el bloque de izquierda (derechas) fue el más votado.

2.1.4. Variables demográficas

Variable	Descripción
Population	Población total del municipio en 2016 (número de empadronados). Relevante tanto para normalizar otras variables como para analizar el efecto del tamaño poblacional sobre el voto.
Age_0-4_Ptge	Porcentaje de ciudadanos con menos de 5 años sobre la población total. Indicador de la tasa de natalidad reciente y del potencial de rejuvenecimiento demográfico del municipio.
Age_under19_Ptge	Porcentaje de ciudadanos menores de 19 años. Refleja el peso del segmento poblacional más joven, que aún no es mayoritariamente elector.
Age_19_65_pct	Porcentaje de ciudadanos con edades comprendidas entre 19 y 65 años. Aproximación a la población en edad de trabajar y en pleno ejercicio de sus derechos políticos.
Age_over65_pct	Porcentaje de ciudadanos mayores de 65 años. Indicador del envejecimiento poblacional, fenómeno particularmente pronunciado en municipios rurales españoles y con implicaciones directas sobre la participación electoral y las preferencias de voto.
WomanPopulationPtge	Porcentaje de mujeres sobre la población total. Permite analizar desequilibrios de género.
ForeignersPtge	Porcentaje de extranjeros sobre la población total. Variable que captura la diversidad migratoria del municipio y que puede estar asociada tanto a dinámicas laborales (agricultura intensiva, turismo) como a variaciones en la composición del censo electoral.
SameComAutonPtge	Porcentaje de ciudadanos que reside en la misma provincia en la que nacieron. Indicador de arraigo territorial y movilidad interna nula.

Variable	Descripción
SameComAutonDiffProvPtge	Porcentaje de ciudadanos que reside en la misma comunidad autónoma en la que nacieron, pero en una provincia diferente. Refleja movilidad intrarregional.
DifComAutonPtge	Porcentaje de ciudadanos que reside en una comunidad autónoma distinta de aquella en la que nacieron. Indicador de movilidad interregional o migración interna de larga distancia.

2.1.5. Variables de desempleo

Variable	Descripción
UnemployLess25_Ptge	Porcentaje de parados menores de 25 años sobre el total de desempleados del municipio. Refleja el peso del desempleo juvenil.
Unemploy25_40_Ptge	Porcentaje de parados con edades comprendidas entre 25 y 40 años sobre el total de desempleados. Representa la franja de trabajadores en edad intermedia, con mayor carga familiar y potencial productivo.
UnemployMore40_Ptge	Porcentaje de parados mayores de 40 años sobre el total de desempleados. Grupo de mayor dificultad para reinserirse en el mercado laboral.

También, tenemos desglosado el desempleo según el sector: agricultura, industria, construcción y servicios.

2.1.6. Variables del tejido empresarial

Variable	Descripción
totalEmpresas	Número total de empresas registradas en el municipio. Indicador de la actividad económica general y del dinamismo empresarial local.
ActividadPpal	Actividad principal del municipio, determinada a partir de cuál de los sectores anteriores concentra mayor número de empresas. Puede tomar los valores: <i>Industria, Construccion, ComercTTEHosteleria, Servicios y Otro</i> .

Encontramos un desglose de este total de empresas en su sector correspondiente: industria, construcción, comercio, transporte y hostelería; y servicios.

2.1.7. Variables territoriales y de evolución demográfica

Variable	Descripción
inmuebles	Número de inmuebles (viviendas y locales) registrados en el municipio.
Pob2010	Población del municipio en 2010. Permite calcular la evolución demográfica en el periodo 2010-2016 y situar el municipio en su trayectoria de crecimiento o declive.

Variable	Descripción
SUPERFICIE	Superficie del municipio expresada en hectáreas. Junto con la población, permite calcular la densidad demográfica real.
Densidad	Densidad de población del municipio, expresada como categoría ordinal: <i>MuyBaja</i> , <i>Baja</i> y <i>Alta</i> . Esta categorización refleja la dicotomía entre España rural y España urbana, particularmente relevante en el debate sobre la despoblación.
PobChange_pct	Porcentaje de cambio en la población respecto al periodo electoral anterior. Los valores negativos indican una disminución de la población.
PersonasInmueble	Número medio de personas que habita un inmueble en el municipio.
Explotaciones	Número de explotaciones agrícolas en el municipio. Permite caracterizar el peso del sector primario en la estructura económica local, con especial relevancia en zonas del interior peninsular y en comunidades de tradición agraria.

2.1.8. Consideraciones sobre la calidad del dato

El dataset presenta algunas peculiaridades dignas de mención. En primer lugar, se observan valores ausentes (*missing values*) en determinadas variables para municipios muy pequeños, posiblemente debido a restricciones de confidencialidad estadística o a la ausencia de registro en la fuente original. En segundo lugar, la variable *Densidad* contiene valores codificados como ? en algunos casos, lo que indica incertidumbre en la clasificación y deberá tratarse adecuadamente en el preprocesado. Asimismo, la variable *Explotaciones* presenta el valor 99999 en ciertos municipios, que actúa previsiblemente como código centinela para datos no disponibles o no aplicables. Por último, algunas variables porcentuales como *Age_over65_pct*, *ForeignersPtge* tienen valores fuera de rango al tomar los porcentajes valores negativos.

Estas circunstancias deberán tenerse en cuenta en la fase de depuración y preparación del dataset antes de proceder con los análisis estadísticos o el modelado predictivo. Una vez analizadas todas las variables que incorpora nuestro dataset, elegiremos las variables **AbstentionPtge** e **Izquierda** como variables objetivo continua y binaria, respectivamente. El resto de variables objetivo las eliminaremos del dataset para no generar más ruido y conseguir un dataset más sencillo tal y como dice el enunciado.

Por tanto, mediante la elección de estas variables, nuestro objetivo con la resolución de este problema es estudiar si el bloque de la izquierda es más potente que el bloque de la derecha mediante una regresión logística y estudiar el porcentaje de abstención, ya que un porcentaje de abstención alto supondría una visión de que el país está muy desarraigado y decepcionado con la política que se está haciendo.

2.2. Importación de datos y asignación correcta de los tipos de variables

Comenzamos el trabajo importando los datos y las librerías. Una vez tenemos los datos cargados, obtenemos los tipos de datos que se han asignado automáticamente a cada una de las variables:

Importación de librerías y dataset

```
1 import os
2 import sys
3 import pickle
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 import pandas as pd
8 import numpy as np
9 import matplotlib.pyplot as plt
10 import seaborn as sns
11 from sklearn.model_selection import train_test_split
12
13 # Aseguramos que FuncionesMineria.py esté en el mismo directorio que este notebook
14 from FuncionesMineria import (
15     analizar_variables_categoricas, cuentaDistintos, frec_variables_num,
16     atipicosAmissing, patron_perdidos, ImputacionCuant, ImputacionQuali,
17     Vcramer, graficoVcramer,
18     mosaico_targetbinaria, boxplot_targetbinaria, hist_targetbinaria,
19     lm, Rsq, validacion_cruzada_lm, modelEffectSizes, crear_data_modelo,
20     lm_forward, lm_backward, lm_stepwise,
21     glm, summary_glm, pseudoR2, validacion_cruzada_glm,
22     impVariablesLog, sensEspCorte, curva_roc,
23     glm_forward, glm_backward, glm_stepwise
24 )
25
26 print("Librerías cargadas correctamente.")
27 # Cargamos el dataset
28 datos = pd.read_excel('DatosEleccionesEspaña.xlsx')
29
30 print(f"Dimensiones: {datos.shape[0]} observaciones x {datos.shape[1]} variables")
```

Como estamos interesados en estudiar las variables *Izquierda* y *AbstentionPctg*, las seleccionamos como variables objetivo y eliminamos el resto de posibles variables objetivo del dataset. Una vez hecho esto, miramos los tipos asignados a cada variable:

Análisis Inicial de Tipos

```
1 # Comprobamos qué variables se han leído como numéricas pero son categóricas
2 # Eliminamos las variables objetivo que NO vamos a utilizar
3 vars_objetivo_eliminar = ['Izda_Pct', 'Dcha_Pct', 'Otros_Pct', 'AbstencionAlta', 'Derecha']
4 datos = datos.drop(vars_objetivo_eliminar, axis=1)
5
6 # Separamos las variables objetivo que SÍ usaremos
7 varObjCont = datos['AbstentionPtge'] # Variable objetivo continua
8 varObjBin = datos['Izquierda'] # Variable objetivo binaria
9
10 # Eliminamos las variables objetivo del conjunto de datos input
11 datos_input = datos.drop(['AbstentionPtge', 'Izquierda'], axis=1)
12
13 # Aunque 'CodigoProvincia' es numérica, realmente es una clasificación de la provincia, por lo que la convertimos a
14 # ↪ string para tratarla como categórica
15 datos_input['CodigoProvincia'] = datos_input['CodigoProvincia'].astype(str)
16
17 # Verificamos y corregimos tipos
18 # 'Densidad' ya viene como object (correcto)
19 # 'CCAA' ya viene como object (correcto)
20 # 'ActividadPpal' ya viene como object (correcto)
21
22 print("Tipos en datos_input tras la corrección:")
23 print(datos_input.dtypes)
```

Así, obtenemos el siguiente output (se ha recortado el output al tratarse de los campos de porcentaje, los cuales se han indicado correctamente como float64):

```

1 Tipos en datos_input tras la corrección:
2 Name          object
3 CodigoProvincia object
4 CCAA          object
5 Population     int64
6 TotalCensus    int64
7 Age_0-4_Ptge   float64
8 Age_under19_Ptge float64
9 Age_19_65_pct  float64
10 ...
11 PobChange_pct  float64
12 PersonasInmuable float64
13 Explotaciones  int64
14 dtype: object

```

De aquí deducimos que tenemos 5 variables categóricas (*Name*, *CodigoProvincia*, *CCAA*, *ActividadPpal*, *Densidad*) siendo el resto de variables numéricas.

2.3. Análisis descriptivo del conjunto de datos

El objetivo de esta sección es estudiar los valores que toman nuestros datos, ser conscientes de su estructura y con ello, detectar posibles errores: valores fuera de rango, datos missing, errores de escritura...

2.3.1. Variables numéricas

Comenzamos haciendo un estudio de las variables numéricas. Para ello, comenzamos viendo el número de valores distintos por si existiese alguna variable numérica con pocos valores que convenga pasar a categórica:

Número de valores distintos en variables numéricas

```

1 # Número de valores distintos por variable numérica
2 # Útil para detectar variables numéricas con pocos valores (candidatas a ser categóricas)
3 distintos = cuentaDistintos(datos_input)
4 print(distintos)

```

	Columna	Distintos
1	Population	3595
2	TotalCensus	3308
3	Age_0-4_Ptge	3759
4	...	...
5	Servicios	756
6	inmuebles	3086
7	Pob2010	3623
8	SUPERFICIE	8109
9	PobChange_pct	3048
10	PersonasInmuable	282
11	Explotaciones	758

Tras la obtención de estos resultados, se ve claramente el amplio rango de valores numéricos que toma cada variable, luego no hay ninguna susceptible de cambiar a categórica. Ahora, estamos en disposición de hacer un análisis de los valores de estas variables para estudiar los rangos de valores y detectar posibles errores. Para ello, utilizamos el siguiente código:

Descriptivo Variables Numéricas

```
1 # --- Variables numéricas ---
2 descriptivos_num = datos_input[numericas_input].describe().T
3 for num in numericas_input:
4     descriptivos_num.loc[num, 'Asimetria'] = datos_input[num].skew()
5     descriptivos_num.loc[num, 'Kurtosis'] = datos_input[num].kurtosis()
6     vals = datos_input[num].dropna().values
7     descriptivos_num.loc[num, 'Rango'] = np.ptp(vals) if len(vals) > 0 else np.nan
8
9 print("Descriptivos variables numéricas:")
10 descriptivos_num.round(4)
```

A partir de los descriptivos obtenidos, vemos que hay porcentajes por debajo de 0 y encima de 100 en los campos *Age_over65_pct* y *ForeignersPctge*, lo cual es claramente un error. Por otro lado, vemos que los campos finales a partir de *totalEmpresas* no tienen 8117 observaciones, lo cual indica valores perdidos. En el caso del campo *Explotaciones*, vemos el valor 99999, el cual llama la atención, probablemente siendo un valor perdido, por lo que lo trataremos más adelante.

Por último, analizando la asimetría y la curtosis de los distintos campos vemos que muchas de ellas están desvirtuadas porque hay mucha diferencia entre las grandes ciudades como Madrid o Barcelona, y los pequeños pueblos de la España rural. Sin embargo, no trataremos esto ya que los consideramos puntos representativos de la población española.

2.3.2. Variables categóricas

Realizamos el estudio de las variables categóricas utilizando la función de FuncionesMineria:

Descriptivo Variables Categóricas

```
1 # --- Variables categóricas ---
2 # Frecuencias de cada categoría para detectar errores de escritura,
3 # categorías poco representadas o missings no declarados
4 resultados_cat = analizar_variables_categoricas(datos_input)
5 for var, tabla in resultados_cat.items():
6     print(f"'{var}':")
7     print(tabla)
8     print()
```

Con los resultados obtenidos, vemos que el campo *Densidad* tiene un valor '?', el cual es un dato missing y representa el 1,13% de las observaciones. Podríamos pensar que los nombres de pueblos con 2 observaciones están duplicados, sin embargo, viendo el dataset nos damos cuenta de que son poblaciones distintas, por lo que no detectamos más errores o cuestiones de análisis en las variables categóricas.

2.4. Corrección de los errores detectados

Tras realizar el análisis descriptivo inicial, se han identificado los siguientes errores y anomalías en el conjunto de datos:

- **Densidad:** Se ha detectado la presencia de la categoría '?', la cual representa un valor perdido (*missing*) no declarado inicialmente.

- **Explotaciones:** Presenta un valor máximo de 99999 en 189 casos. Dada la naturaleza de la variable, este valor se identifica como un código para valores perdidos no declarados.
- **Variables con valores perdidos reales (NaN):** Las variables `totalEmpresas`, `Industria`, `Construccion`, `ComercTTEHosteleria`, `Servicios`, `inmuebles`, `Pob2010`, `SUPERFICIE`, `PobChange_pct` y `PersonasInmueble` presentan valores nulos que requieren tratamiento.
- **Age_over65_pct y ForeignersPtge:** Se han identificado registros con porcentajes fuera del rango lógico (0, 100), lo que indica errores en la medición o entrada de datos.

Por otro lado, no se detectan errores de escritura o inconsistencias en las variables categóricas (`Name`, `CodigoProvincia`, `CCAA` y `ActividadPpal`). Asimismo, la variable objetivo continua (`AbstentionPtge`) presenta valores consistentes dentro del rango esperado (0-100).

Para corregirlos, hacemos los siguientes ajustes:

Corrección de errores

```
1 # 1. Missings no declarados en variable categórica: Densidad → '?'
2 datos_input['Densidad'] = datos_input['Densidad'].replace('?', np.nan)
3
4 # 2. Missings no declarados en variable numérica: Explotaciones → 99999
5 datos_input['Explotaciones'] = datos_input['Explotaciones'].replace(99999, np.nan)
6
7 # Valores fuera de rango
8 datos_input['Age_over65_pct'] = [x if 0 <= x <= 100 else np.nan for x in
  ↳ datos_input['Age_over65_pct']]
9 datos_input['ForeignersPtge'] = [x if 0 <= x <= 100 else np.nan for x in
  ↳ datos_input['ForeignersPtge']]
```

2.5. Análisis de valores atípicos

Por definición, decimos que un *dato atípico* es una observación numéricamente distante del resto de datos. Tras el análisis previo, hemos visto que la mayoría de variables no tienen una distribución simétrica, por lo que no nos sirve el criterio de la desviación típica para detectar los *outliers*.

Utilizaremos los criterios **MAD** y del **Rango Intercuartílico** para detectar los valores atípicos. Para ello, utilizaremos la función `atipicosAmissing`, la cual aplica ambos métodos y solo clasifica el dato como atípico en caso de que se cumplan ambos criterios. De esta forma, tenemos el siguiente código:

Detección de valores atípicos

```
1 # Calculamos el porcentaje de atípicos por variable numérica
2 resultados_atip = {x: atipicosAmissing(datos_input[x])[1] / len(datos_input)
  ↳ for x in numericas_input}
3
4
5 print("Porcentaje de atípicos por variable numérica:")
6 for var, pct in sorted(resultados_atip.items(), key=lambda x: -x[1]):
7     if pct > 0:
8         print(f" {var}: {pct:.4f} ({pct*100:.2f}%)")
9
10 # Convertimos los atípicos a missings
11 for x in numericas_input:
12     datos_input[x] = atipicosAmissing(datos_input[x])[0]
13
14 print("Missings por variable tras tratamiento de atípicos:")
15 print(datos_input.isna().sum())
```

Obteniendo los siguientes resultados:

```

1 Porcentaje de atípicos por variable numérica:
2 Population: 0.0991 (9.91%)
3 TotalCensus: 0.0963 (9.63%)
4 SameComAutonDiffProvPtge: 0.0203 (2.03%)
5 ...
6 WomanPopulationPtge: 0.0026 (0.26%)
7 SameComAutonPtge: 0.0001 (0.01%)
8
9 Missings por variable tras tratamiento de atípicos:
10 Name 0
11 CodigoProvincia 0
12 CCAA 0
13 Population 804
14 TotalCensus 782
15 Age_0-4_Ptge 0
16 Age_under19_Ptge 0
17 Age_19_65_pct 24
18 ...
19 totalEmpresas 5
20 Industria 188
21 Construcccion 139
22 ...
23 PobChange_pct 7
24 PersonasInmueble 138
25 Explotaciones 189
26 dtype: int64
27

```

2.6. Análisis de valores perdidos

Una vez hemos obtenido y clasificado todos los datos faltantes, esta ausencia hay que analizarla para tratar de reducir el perjuicio que puede provocar esto en el análisis estadístico posterior. Tendremos que analizar detalladamente los *missings* para decidir la estrategia a seguir: eliminación, recategorización o imputación.

Para hacer este análisis primero echamos un primer vistazo con el mapa de correlaciones de valores perdidos:

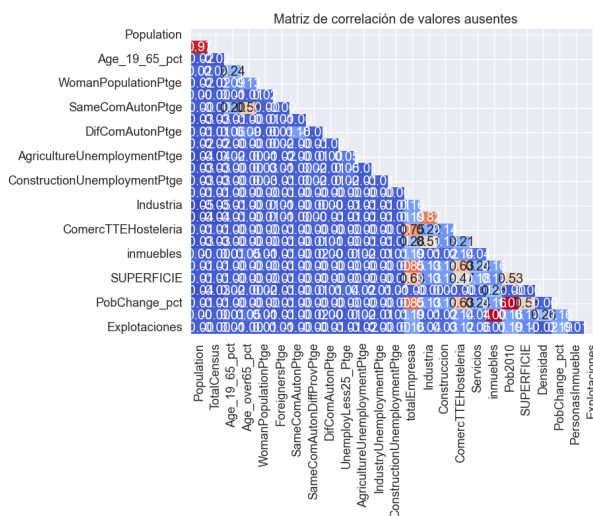


Figura 2.1: Patrón de perdidos.

Una vez hecho esto, debemos tomar una decisión sobre el método de arreglo de *missings* por lo que estudiaremos si hay variables que tienen más del 50% de observaciones perdidas:

Porcentaje de valores perdidos por variable

```
1 # Proporción de missings por variable
2 prop_missingsVars = datos_input.isna().sum() / len(datos_input)
3 print("Proporción de missings por variable:")
4 print(prop_missingsVars[prop_missingsVars > 0].sort_values(ascending=False).round(4))
```

Con esto, observamos que no hay ninguna variable que supere el 50% de valores perdidos, luego no eliminamos ninguna variable (por ello, no mostramos el código que se usaría para eliminar variables en caso de que existieran). Asimismo, como el número de valores distintos en la proporción de *missings* es pequeño (8), creamos una nueva variable **propMissings** que almacene el número de *missings* en cada observación. Como no hay ninguna observación con más del 50% de variables perdidas, mantenemos todas las observaciones también. Por tanto, no procedemos con la eliminación en ningún caso.

El siguiente paso es estudiar la recategorización. La única variable de ser susceptible de sufrir esto es *Densidad* al tener un 4,4% de *missings*, luego podremos tratarla como categoría adicional y posteriormente la imputaremos como variable cualitativa. Finalmente, reemplazaremos los valores faltantes como imputaciones. Por tanto, para terminar con esta sección aplicaremos a todos los datos que continúan *missing* la imputación aleatoria:

Porcentaje de valores perdidos por variable

```
1 # IMPUTACIONES
2 # Variables cuantitativas: imputación aleatoria
3 for x in numericas_input:
4     datos_input[x] = ImputacionCuant(datos_input[x], 'aleatorio')
5
6 # Variables cualitativas: imputación aleatoria
7 for x in categoricas_input:
8     datos_input[x] = ImputacionCuali(datos_input[x], 'aleatorio')
9
10 # Verificamos que no quedan missings
11 print("Missings tras imputación:")
12 print(datos_input.isna().sum().sum())
```

Obtenemos así una cantidad de 0 valores missing tras la imputación y guardamos los datos depurados para proceder con la elaboración de los modelos.

3 Regresión Lineal

Comenzamos el capítulo destinado a la elaboración del modelo de regresión lineal con el objetivo de predecir la variable objetivo **AbstentionPtge**. Para ello, utilizaremos los métodos de selección clásica de variables (Forward, Backward y Stepwise mediante los criterios AIC y BIC) sin transformaciones ni interacciones (tal y como indica el enunciado). Lo primero que haremos será cargar el dataset depurado para su posterior análisis.

Algo útil en la elaboración del modelo es el uso del estadístico V de Cramer, el cual evalúa la relación (princiaplamente útil para ver la no lineal) entre cualesquiera dos variables. Por tanto, es lo primero que mostraremos para orientarnos a la hora de crear nuestro modelo:

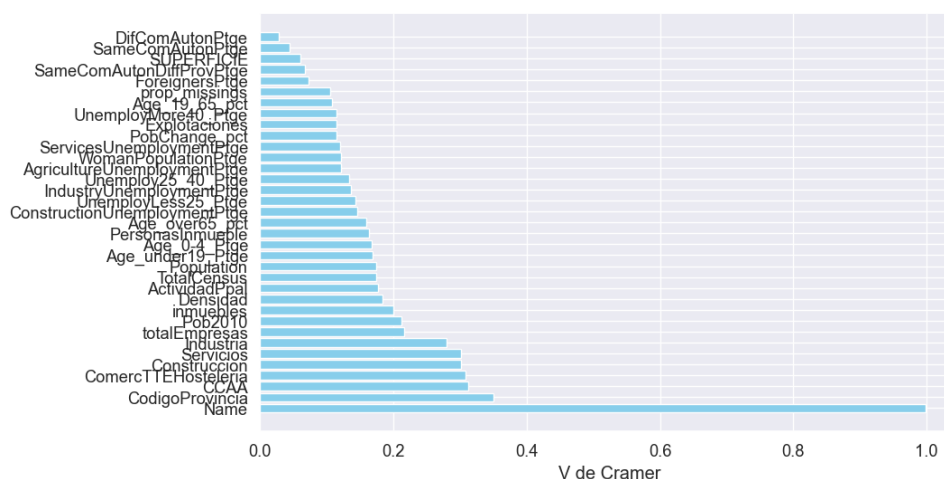


Figura 3.1: Gráfico V de Cramer.

Vemos que la variable *Name* es la que mayor relación tiene, sin embargo, consideramos que tiene dos grandes problemas. El estadístico V de Cramer para esa variable toma el valor 1, luego es prácticamente una relación perfecta, lo que supondría, total explicabilidad, cayendo así en el *overfitting* (tiene sentido que tome este valor al ser prácticamente un índice para cada observación). Por otro lado, hay 8117 nombres de pueblos en el dataset, lo que supondría que, al introducirla en el modelo, se crearían 8116 variables *dummy*, lo que haría al modelo muy pesado e intratable. Por eso, decidimos eliminar la variable *Name* de nuestro conjunto de datos.

A continuación, preparamos nuestro conjunto de datos para la elaboración del modelo. Para ello, dividiremos nuestro dataset en el conjunto de entrenamiento y el conjunto de prueba. En este caso, hemos hecho una partición 70 % y 30 % respectivamente, por la peculiaridad del problema. También, tuvimos que encontrar una semilla que recogiese bien todas las particularidades de los datos del conjuntos para

que al crear variables dummy, se creasen todas y no diese error al calcular las distintas métricas. De esta manera, nuestro código de división es el siguiente:

División Train y Test

```
1 # División train / test (70% / 30%)
2 x_train, x_test, y_train, y_test = train_test_split(
3     datos_input, varObjCont, test_size=0.3, random_state=12345678)
4
5 print(f"Train: {x_train.shape[0]} observaciones")
6 print(f"Test : {x_test.shape[0]} observaciones")
7
8 # Variables continuas y categóricas para los modelos
9 var_cont = numericas_input.copy()
10 var_categ = categoricas_input.copy()
```

Comenzamos con el entrenamiento del modelo. Para ello, seguimos las indicaciones del enunciado: NO hacemos interacciones entre variables ni transformaciones y aplicamos los métodos de selección clásica de variables: Forward, Backward y Stepwise.

Mostraremos el código de uno de estos métodos (por la restricción en la longitud de la entrega) y para los siguientes se supondrá visto al seguir la misma estructura y solo cambiar los nombres de los métodos o de la técnica de entrenamiento.

Creación del modelo

```
1 # Forward AIC
2 print("=" * 60)
3 print("FORWARD AIC")
4 print("=" * 60)
5 modeloForwAIC = lm_forward(y_train, x_train, var_cont, var_categ, [], 'AIC')
6 R2_forwAIC_train = Rsq(modeloForwAIC['Modelo'], y_train, modeloForwAIC['X'])
7 x_test_forwAIC = crear_data_modelo(x_test,
8     modeloForwAIC['Variables']['cont'],
9     modeloForwAIC['Variables']['categ'],
10    modeloForwAIC['Variables']['inter'])
11 R2_forwAIC_test = Rsq(modeloForwAIC['Modelo'], y_test, x_test_forwAIC)
12 npar_forwAIC = len(modeloForwAIC['Modelo'].params)
13 print(f"Forward AIC → R² Train: {R2_forwAIC_train:.4f} | R² Test: {R2_forwAIC_test:.4f} |
    → N° params: {npar_forwAIC}")
```

Tras realizar el entrenamiento y el test, obtenemos el siguiente cuadro de resultados:

	Método	R ² Train	R ² Test	N° Parámetros
1	Forward AIC	0.4321	0.4241	80
2	Forward BIC	0.4237	0.4180	62
3	Backward AIC	0.4329	0.4244	81
4	Backward BIC	0.4257	0.4214	65
5	Stepwise AIC	0.4320	0.4239	79
6	Stepwise BIC	0.4237	0.4180	62

Tras aplicar la validación cruzada, separando en 5 bloques de datos con 20 repeticiones cada uno:


```

1 # Validación cruzada repetida (20 repeticiones x 5 folds)
2 results_lm = pd.DataFrame({'Rsquared': [], 'Resample': [], 'Modelo': []})
3
4 for rep in range(20):
5     m1VC = validacion_cruzada_lm(5, x_train, y_train,
6                                 modeloForwAIC['Variables']['cont'], modeloForwAIC['Variables']['categ'],
7                                 modeloForwAIC['Variables']['inter'])
8     m2VC = validacion_cruzada_lm(5, x_train, y_train,
9                                 modeloForwBIC['Variables']['cont'], modeloForwBIC['Variables']['categ'],
10                                modeloForwBIC['Variables']['inter'])
11     m3VC = validacion_cruzada_lm(5, x_train, y_train,
12                                modeloBackAIC['Variables']['cont'], modeloBackAIC['Variables']['categ'],
13                                modeloBackAIC['Variables']['inter'])
14     m4VC = validacion_cruzada_lm(5, x_train, y_train,
15                                modeloBackBIC['Variables']['cont'], modeloBackBIC['Variables']['categ'],
16                                modeloBackBIC['Variables']['inter'])
17     m5VC = validacion_cruzada_lm(5, x_train, y_train,
18                                modeloStepAIC['Variables']['cont'], modeloStepAIC['Variables']['categ'],
19                                modeloStepAIC['Variables']['inter'])
20     m6VC = validacion_cruzada_lm(5, x_train, y_train,
21                                modeloStepBIC['Variables']['cont'], modeloStepBIC['Variables']['categ'],
22                                modeloStepBIC['Variables']['inter'])
23
24     rep_df = pd.DataFrame({
25         'Rsquared': m1VC + m2VC + m3VC + m4VC + m5VC + m6VC,
26         'Resample': ['Rep' + str(rep+1)] * 5 * 6,
27         'Modelo': [1]*5 + [2]*5 + [3]*5 + [4]*5 + [5]*5 + [6]*5
28     })
29     results_lm = pd.concat([results_lm, rep_df], axis=0)
30
31 print("Validación cruzada completada.")

```

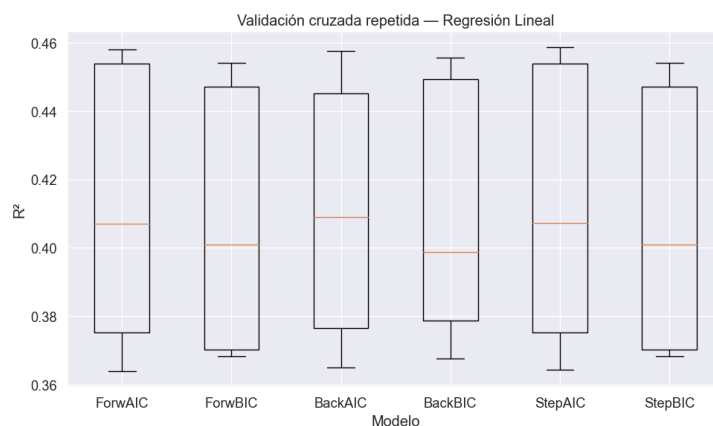


Figura 3.2: Validación Cruzada LM.

1 Media R^2 por modelo:	Desv. típica R^2 por modelo:
2 Modelo	Modelo
3 1.0 0.4116	1.0 0.0391
4 2.0 0.4081	2.0 0.0368
5 3.0 0.4107	3.0 0.0367
6 4.0 0.4100	4.0 0.0363
7 5.0 0.4119	5.0 0.0391
8 6.0 0.4081	6.0 0.0368

Entonces, una vez obtenidos todos los modelos, tendremos que compararlos para tomar una decisión sobre cuál tomar. Esta decisión es un conjunto de selecciones basadas entre el mejor equilibrio del R^2 , la mínima desviación típica y la parsimonia (mínimo número de parámetros). De forma general, vemos que todos los métodos obtenidos tienen un comportamiento bastante similar con R^2 y desviaciones con

diferencias del orden de milésimas. Además, vemos que el ajuste lineal no es perfecto para explicar esta relación ya que en general, en la práctica decimos que un modelo es bueno cuando el R^2 se sitúa sobre $0,6 - 0,7$. Por tanto, vemos que todos los modelos basados en el AIC, tienen R^2 más altos, sin embargo, tienen una cantidad de parámetros desproporcionados, luego quedan descartados. Por tanto, tomaremos el modelo Forward BIC, coincidente con el modelo Stepwise BIC ya que su diferencia con el Backward BIC es muy pequeña y tiene 3 parámetros menos.

1	OLS Regression Results							
2	=====							
3	Dep. Variable:	AbstentionPtge	R-squared:	0.424				
4	Model:	OLS	Adj. R-squared:	0.417				
5	Method:	Least Squares	F-statistic:	67.74				
6	Date:	Thu, 14 May 2026	Prob (F-statistic):	0.00				
7	Time:	07:05:03	Log-Likelihood:	-17952.				
8	No. Observations:	5681	AIC:	3.603e+04				
9	Df Residuals:	5619	BIC:	3.644e+04				
10	Df Model:	61						
11	Covariance Type:	nonrobust						
12	=====							
13		coef	std err	t	P> t	[0.025	0.975]	
14	-----							
15	const	43.2454	1.927	22.446	0.000	39.468	47.022	
16	SameComAutonPtge	-0.0883	0.008	-10.573	0.000	-0.105	-0.072	
17	Explotaciones	0.0029	0.000	6.628	0.000	0.002	0.004	
18	ConstructionUnemploymentPtge	0.0480	0.007	6.603	0.000	0.034	0.062	
19	Age_0-4_Ptge	0.2835	0.055	5.186	0.000	0.176	0.391	
20	Age_19_65_pct	-0.0772	0.015	-5.309	0.000	-0.106	-0.049	
21	ServicesUnemploymentPtge	0.0166	0.004	4.722	0.000	0.010	0.024	
22	IndustryUnemploymentPtge	0.0331	0.008	4.018	0.000	0.017	0.049	
23	WomanPopulationPtge	-0.0682	0.022	-3.071	0.002	-0.112	-0.025	
24	CodigoProvincia_10	-3.2873	1.111	-2.958	0.003	-5.466	-1.109	
25	CodigoProvincia_11	4.3720	1.470	2.973	0.003	1.490	7.254	
26	CodigoProvincia_12	-9.1211	1.153	-7.913	0.000	-11.381	-6.861	
27	CodigoProvincia_13	-5.8117	1.218	-4.772	0.000	-8.199	-3.424	
28	CodigoProvincia_14	-4.9464	1.304	-3.793	0.000	-7.503	-2.390	
29	...							
30	CodigoProvincia_8	2.4763	1.073	2.308	0.021	0.373	4.580	
31	CodigoProvincia_9	-4.5559	1.059	-4.302	0.000	-6.632	-2.480	
32	ActividadPpal_Construccion	-3.3435	1.831	-1.826	0.068	-6.932	0.245	
33	ActividadPpal_Industria	0.6952	1.933	0.360	0.719	-3.095	4.485	
34	ActividadPpal_Utro	-2.2612	0.240	-9.412	0.000	-2.732	-1.790	
35	ActividadPpal_Servicios	-0.9376	0.334	-2.809	0.005	-1.592	-0.283	
36	=====							
37	Omnibus:	242.242	Durbin-Watson:	1.982				
38	Prob(Omnibus):	0.000	Jarque-Bera (JB):	706.775				
39	Skew:	0.151	Prob(JB):	3.35e-154				
40	Kurtosis:	4.701	Cond. No.	2.40e+04				
41	=====							
42								

De estos resultado, comprobamos que los residuos no siguen una distribución normal (hipótesis rechazada por el estadístico de Jarque-Bera) pero sí están incorrelados (Durbin-Watson). A partir de las variables del modelo y de los coeficientes obtenidos mediante el entrenamiento, podemos entender el modelo de la siguiente manera (seleccionaremos una variable categórica y otra numérica para ilustrar la interpretación):

- **Variable continua ConstructionUnemploymentPtge (porcentaje de desempleo en la Construcción):** Sea el coeficiente 0.048, significa que por cada punto porcentual adicional de desempleo en el sector de la construcción, el porcentaje de abstención aumenta en 0.048 puntos porcentuales dejando el resto de variables constantes. Es decir, los municipios con más paro en la construcción tienden a tener ligeramente mayor abstención. Esto podría explicarse por el hartazgo de la población en desempleo en este sector con la política y la falta de recursos que se destinan a este sector.
- **Variable categórica ActividadPpal:** El coeficiente de, por ejemplo, 'ActividadPpal_Servicios' respecto a la categoría de referencia (ComercTTEHosteleria) indica la diferencia media en abstención entre municipios cuya actividad principal son los servicios y la categoría base, *ceteris paribus*.

Finalmente, vemos la importancia de las variables que han entrado dentro del modelo con su aportación a la explicabilidad del modelo con el R^2 :

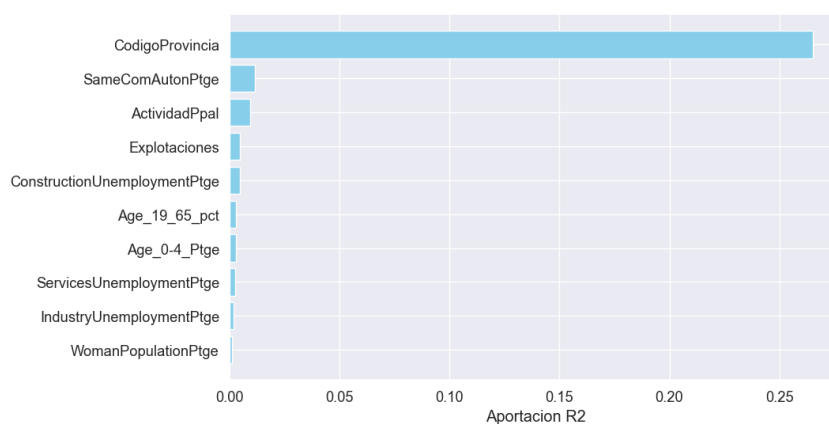


Figura 3.3: Aportación de Variables.

Vemos una clara dominancia del *CodigoProvincia*, lo cual tiene sentido ya que históricamente siempre ha habido un predominio ideológico en cada provincia en función del sector al que estuviese dedicada dicha provincia. Además, viendo el resto de variables que entran del modelo, vemos que el desempleo es algo que se tienen muy en cuenta dentro de la política, principalmente en el sector de la construcción y más en 2016 cuando en ese sector no se estaba invirtiendo tras el colapso del sector por la crisis financiera.

4 Regresión Logística

Finalmente, llegamos al último capítulo de la entrega: la construcción del modelo de regresión logística. En este apartado, hemos decidido construir un modelo para la predicción de la variable binaria *Izquierda*, donde toma el valor 1 si hay más votos de izquierda que de derechas y otros; o 0 en otro caso. Debido a las características de esta variable binaria, no podemos aplicar una regresión lineal clásica, ya que obtendríamos valores no acotados, lo cual no tiene sentido para este problema de clasificación. Por ello, recurrimos a la regresión logística, la cual nos da la probabilidad de que la variable *Izquierda* tome cada uno de los valores.

El procedimiento a seguir es prácticamente análogo al seguido en el capítulo anterior para la regresión lineal. Sin embargo, debido al nuevo enfoque, no nos vale con quedarnos en la predicción, ya que esta solo nos da la probabilidad. Tendremos que interpretar esta probabilidad para tomar decisiones sobre donde clasificar nuestras predicciones y para ello, también cambiarán nuestras métricas para evaluar el ajuste: emplearemos el *pseudo- R^2* y la curva ROC para medir la sensibilidad frente a la tasa de falsos positivos.

Comenzamos la construcción del modelo dividiendo nuestro conjunto de datos de manera aleatoria en conjunto de entrenamiento y de test. De nuevo, hemos ampliado el tamaño del test respecto del utilizado en clase para tener una muestra que tomase todos los valores de las variables, de forma que fuese representativa del conjunto y no diese problemas posteriormente al no crear todas las variables dummy:

División Train y Test

```
1 # División train / test (mismo random_state que en regresión lineal para comparabilidad)
2 x_train_b, x_test_b, y_train_b, y_test_b = train_test_split(
3     datos_input, varObjBin, test_size=0.3, random_state=12345678)
4
5 y_train_b = y_train_b.astype(int)
6 y_test_b = y_test_b.astype(int)
7
8 print(f"Train: {x_train_b.shape[0]} | Test: {x_test_b.shape[0]}")
9
10 Train: 5681 | Test: 2436
```

Para la construcción del modelo seguimos los pasos del capítulo anterior, con los métodos de selección clásica de variables: Forward, Backward y Stepwise, basados tanto en el AIC como en el BIC. Tras el entrenamiento, obtenemos los siguientes resultados a partir del código que se muestra a continuación (de nuevo, mostraremos solo el código para uno de los métodos, ya que es generalizable al resto):

Entrenamiento modelo de Regresión Logística

```

1 # Forward AIC
2 print("=" * 60)
3 print("GLM FORWARD AIC")
4 print("=" * 60)
5 modeloGLM_forwAIC = glm_forward(y_train_b, x_train_b, var_cont, var_categ, [], 'AIC')
6
7 pR2_glm_forwAIC_train = pseudoR2(modeloGLM_forwAIC['Modelo'], modeloGLM_forwAIC['X'],
  ↪ y_train_b)
8 x_test_glm_forwAIC = crear_data_modelo(x_test_b,
9                                     modeloGLM_forwAIC['Variables']['cont'],
10                                    modeloGLM_forwAIC['Variables']['categ'],
11                                    modeloGLM_forwAIC['Variables']['inter'])
12 pR2_glm_forwAIC_test = pseudoR2(modeloGLM_forwAIC['Modelo'], x_test_glm_forwAIC,
  ↪ y_test_b)
13 npar_glm_forwAIC = len(modeloGLM_forwAIC['Modelo'].coef_[0])
14 print(f"GLM Forward AIC → pR2 Train: {pR2_glm_forwAIC_train:.4f} | pR2 Test:
  ↪ {pR2_glm_forwAIC_test:.4f} | N° params: {npar_glm_forwAIC}")

```

Método	pR ² Train	pR ² Test	N° Parámetros
Forw AIC	0.3325	0.3490	78
Forw BIC	0.3325	0.3490	78
Back AIC	0.3293	0.3405	74
Back BIC	0.3293	0.3405	74
Step AIC	0.3314	0.3479	76
Step BIC	0.3314	0.3479	76

Tras esta primera construcción, vemos que todos comparten un elevado número de parámetros y valores del $pseudo-R^2$ muy similares. También, destacar que se han obtenido buenos valores para este índice (tal y como vimos en clase, entre 0.2 y 0.4, se consideraban buenos valores). Por tanto, hemos obtenido buenos modelos para la explicabilidad de esta variable.

Procedemos con la elección del modelo ganador y como en regresión lineal, recurrimos a la validación cruzada para estudiar el mejor modelo. Para la selección, emplearemos el criterio del área bajo la curva ROC. Tras la validación cruzada, estos son los resultados obtenidos:

Validación Cruzada para Regresión Logística

```

1 # Validación cruzada repetida para regresión logística
2 results_glm = pd.DataFrame({'AUC': [], 'Resample': [], 'Modelo': []})
3
4 for rep in range(20):
5     m1VC = validacion_cruzada_glm(5, x_train_b, y_train_b,
6                                   modeloGLM_forwAIC['Variables']['cont'],
7                                   ↪ modeloGLM_forwAIC['Variables']['categ'],
8                                   modeloGLM_forwAIC['Variables']['inter'])
9     m2VC = validacion_cruzada_glm(5, x_train_b, y_train_b,
10                                   modeloGLM_forwBIC['Variables']['cont'],
11                                   ↪ modeloGLM_forwBIC['Variables']['categ'],
12                                   modeloGLM_forwBIC['Variables']['inter'])

```

Validación Cruzada para Regresión Logística

```

1  m3VC = validacion_cruzada_glm(5, x_train_b, y_train_b,
2      modeloGLM_backAIC['Variables']['cont'],
3      ↪ modeloGLM_backAIC['Variables']['categ'],
4      modeloGLM_backAIC['Variables']['inter'])
5  m4VC = validacion_cruzada_glm(5, x_train_b, y_train_b,
6      modeloGLM_backBIC['Variables']['cont'],
7      ↪ modeloGLM_backBIC['Variables']['categ'],
8      modeloGLM_backBIC['Variables']['inter'])
9  m5VC = validacion_cruzada_glm(5, x_train_b, y_train_b,
10     modeloGLM_stepAIC['Variables']['cont'],
11     ↪ modeloGLM_stepAIC['Variables']['categ'],
12     modeloGLM_stepAIC['Variables']['inter'])
13 m6VC = validacion_cruzada_glm(5, x_train_b, y_train_b,
14     modeloGLM_stepBIC['Variables']['cont'],
15     ↪ modeloGLM_stepBIC['Variables']['categ'],
16     modeloGLM_stepBIC['Variables']['inter'])
17
18 rep_df = pd.DataFrame({
19     'AUC':      m1VC + m2VC + m3VC + m4VC + m5VC + m6VC,
20     'Resample': ['Rep' + str(rep+1)] * 5 * 6,
21     'Modelo':   [1]*5 + [2]*5 + [3]*5 + [4]*5 + [5]*5 + [6]*5
22 })
23 results_glm = pd.concat([results_glm, rep_df], axis=0)
24
25 print("Validación cruzada GLM completada.")

```

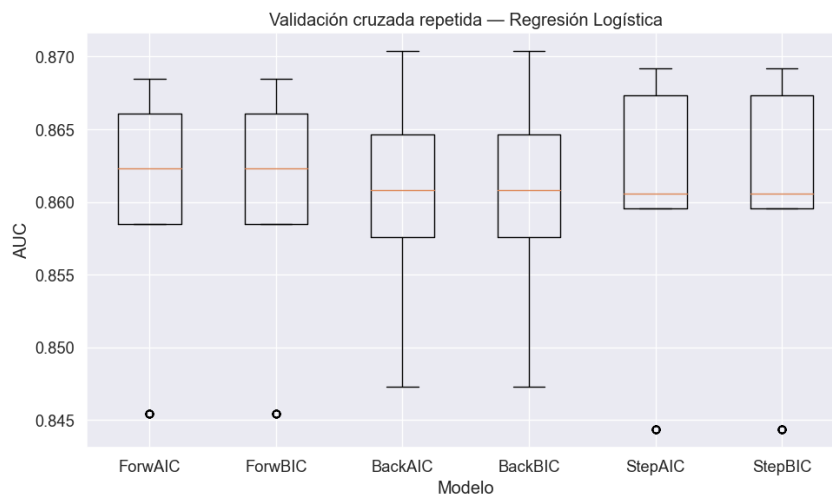


Figura 4.1: Validación Cruzada Regresión Logística.

Algo que podemos observar y que también podríamos haber deducido con los resultados anteriores es que, en este caso, el método de selección de variables selecciona el mismo modelo independientemente del criterio utilizado (AIC o BIC). Para verlo numéricamente y no tomar la decisión únicamente con el gráfico, mostramos los resultados obtenidos a continuación:

```

1 Media AUC por modelo:           Desv. típica AUC por modelo:
2 Modelo                         Modelo
3 1.0      0.8601                1.0      0.0082
4 2.0      0.8601                2.0      0.0082
5 3.0      0.8601                3.0      0.0077
6 4.0      0.8601                4.0      0.0077
7 5.0      0.8602                5.0      0.0088
8 6.0      0.8602                6.0      0.0088
9 Name: AUC, dtype: float64

```

Viendo estos resultados, consideramos que el mejor modelo en AUC es el 5 (coincidente con el 6) y a pesar de que sea el de mayor desviación típica, es una desviación típica ínfima con una diferencia de milésimas. Por otro lado, en cuanto al número de variables, todas contienen un número similar, por lo que seleccionamos un punto medio, el correspondiente al método Stepwise. Por tanto, bajo todos estos criterios, optamos por el modelo 5.

```

1 === Contrastes de significación ===
2           Variable Estimate Std. Error      z value p value signif
3           (Intercept) -2.210015         NaN         NaN      nan
4           DifComAutonPtge 0.030183 5.489149e-03 5.498643e+00 0.000000    ***
5           Explotaciones -0.000955 2.300271e-04 -4.150525e+00 0.000034    ***
6           UnemployLess25_Ptge 0.020050 4.947721e-03 4.052347e+00 0.000051    ***
7           Age_19_65_pct 0.031095 7.360231e-03 4.224790e+00 0.000024    ***
8           Age_0-4_Ptge -0.081259 2.788265e-02 -2.914306e+00 0.003579    **
9 AgricultureUnemploymentPtge 0.015207 4.517447e-03 3.366214e+00 0.000767    ***
10          ForeignersPtge -0.016893 6.573858e-03 -2.569759e+00 0.010203     *
11          ...
12          CodigoProvincia_10 -0.017483         NaN         NaN      nan
13          CodigoProvincia_11 1.370628         NaN         NaN      nan
14          CCAA_Rioja -0.651243 7.197929e+07 -9.047644e-09 1.000000
15
16 === Bondad de ajuste ===
17          LLK          AIC          BIC
18 -2021.721705 4047.44341 4060.733175

```

Terminamos esta sección evaluando el modelo. Para ello, las dos medidas de evaluación que utilizaremos será el *pseudo-R²* y su comparación con el conjunto de test y la curva ROC:

- **Pseudo-R²**: Una vez seleccionado el modelo ganador, evaluamos su estabilidad para ver si hemos caído en el sobreajuste.

```

1 Optimization terminated successfully.
2           Current function value: 0.532242
3           Iterations 5
4 Optimization terminated successfully.
5           Current function value: 0.526430
6           Iterations 5
7 Pseudo-R2 Train: 0.3314
8 Pseudo-R2 Test : 0.3479
9 Diferencia      : -0.0166
10 Número de parámetros: 76

```

Con los resultados obtenidos, confirmamos que tenemos un buen ajuste del modelo basándonos en el valor del *pseudo-R²* y su mejor rendimiento en el conjunto test.

- **Curva ROC:** Esta curva es un método para evaluar el modelo sin necesidad de clasificar cada observación como evento o no evento. Muestra la sensibilidad frente a la tasa de falsos positivos para todos los posibles valores de corte de la probabilidad a posteriori.

El modelo perfecto es aquel que otorga probabilidad 0 a los no eventos y 1, a los eventos. Luego, existe un corte de la probabilidad que clasifica correctamente todas las observaciones dando lugar a la sensibilidad y especificidad de 1. En caso de encontrar esta probabilidad, la curva formaría un ángulo recto, luego cuanto más parecida a los lados de un triángulo rectángulo respecto de la bisectriz, mejor será el ajuste. Este modelo tendría área 1, al coincidir con el área del cuadrado.

Dicho esto, nuestro modelo obtiene los siguientes resultados:

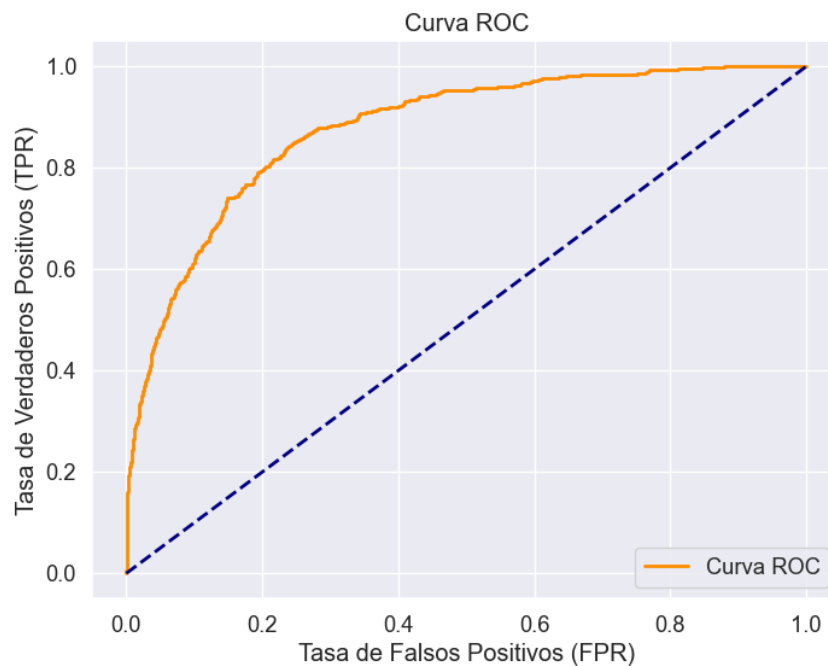


Figura 4.2: Curva ROC.

```
1 # Curva ROC del modelo ganador
2 AUC_ganador = curva_roc(x_test_ganador_glm, y_test_b, ModeloGanadorGLM)
3 Área bajo la curva ROC = 0.8786688757024095
```

A partir de ese valor del área bajo la curva, vemos que es bastante cercano a 1, luego podemos reiterarnos en nuestra hipótesis de que hemos tenido un buen ajuste del modelo y se comporta bien en la mayoría de casos.

4.1. Determinación del punto de corte óptimo

Para la selección del punto de corte óptimo, podemos seguir varias estrategias y seleccionaremos aquella que mejor se adapte al objetivo que buscamos con nuestro análisis. Para la elaboración de la tarea, hemos considerado que no solo estamos interesados en maximizar la tasa de aciertos, si no que nos interesa más maximizar las predicciones de los municipios donde ganará la izquierda (sensibilidad) y donde no ganará (especificidad); luego, utilizaremos el índice de Youden como métrica para seleccionar el punto de corte.

Esto es porque al tratarse de un tema político, un par de pueblos o ciudades pueden decidir el resultado final de las elecciones, tal y como ocurre con los estados bisagra en EEUU.

```

1  # Generamos una rejilla de puntos de corte entre 0 y 1
2  posiblesCortes = np.arange(0, 1.01, 0.01).tolist()
3  rejilla = pd.DataFrame({'PtoCorte':[], 'Accuracy':[], 'Sensitivity':[],
4                          'Specificity':[], 'PosPredValue':[], 'NegPredValue':[]})
5
6  for pto in posiblesCortes:
7      rejilla = pd.concat([
8          rejilla,
9          sensEspCorte(ModeloGanadorGLM['Modelo'], x_test_b, y_test_b, pto,
10                      ModeloGanadorGLM['Variables']['cont'],
11                      ModeloGanadorGLM['Variables']['categ'],
12                      ModeloGanadorGLM['Variables']['inter'])
13      ], axis=0)
14
15  rejilla['Youden'] = rejilla['Sensitivity'] + rejilla['Specificity'] - 1
16  rejilla.index = range(len(rejilla))

```

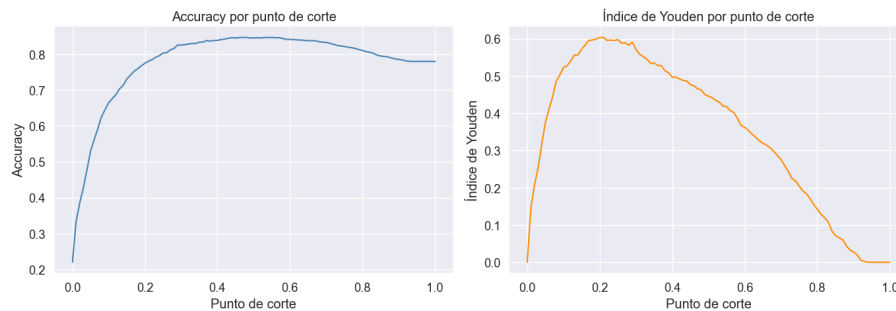


Figura 4.3: Comparativa puntos de corte.

A partir de las representaciones, obtenemos los siguientes resultados:

```

1  Punto de corte óptimo (Accuracy): 0.48
2  Punto de corte óptimo (Youden)  : 0.21
3
4  === Métricas con punto de corte de máxima Accuracy ===
5  PtoCorte Accuracy Sensitivity Specificity PosPredValue NegPredValue
6      0.48  0.847701    0.523364    0.938979    0.707071    0.875
7
8  === Métricas con punto de corte de máximo Youden ===
9  PtoCorte Accuracy Sensitivity Specificity PosPredValue NegPredValue
10     0.21  0.781199    0.839252    0.764861    0.501116    0.944156

```

Por tanto, reiteramos que nuestro objetivo es equilibrar tanto los casos positivos como los casos negativos, por lo que usaremos el **índice de Youden**. A continuación, mostramos cómo funciona tanto en el conjunto test como en el de entrenamiento. Vemos que tiene comportamientos similares e incluso mejora en el conjunto test:

```

1 === Evaluación final con punto de corte = 0.21 ===
2 Train:
3 PtoCorte Accuracy Sensitivity Specificity PosPredValue NegPredValue
4 0.21 0.767823 0.83281 0.749036 0.489617 0.939385
5
6 Test:
7 PtoCorte Accuracy Sensitivity Specificity PosPredValue NegPredValue
8 0.21 0.781199 0.839252 0.764861 0.501116 0.944156

```

4.2. Importancia de las variables en el modelo

Terminamos la optimización del modelo para calcular la importancia de las variables en el modelo. Para ello, utilizaremos la función de FuncionesMineria `impVariablesLog` que calculará la importancia de las variables respecto al pseudo- R^2 . Así, obtenemos los siguientes resultados en cuanto a su aporte al pseudo- R^2 :

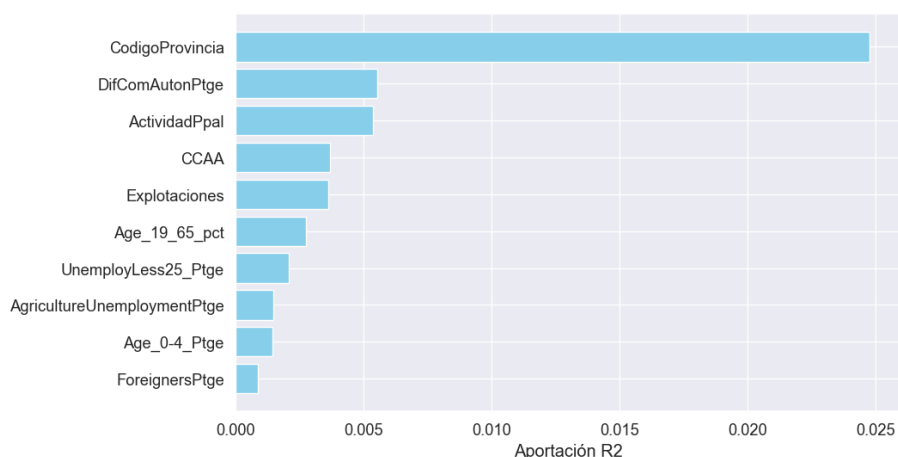


Figura 4.4: Importancia de las variables en el modelo.

De nuevo, vemos la importancia de la provincia, lo cual es coherente, ya que muchas provincias tienen connotaciones históricas en sus ideas políticas según su sector principal de trabajo y el desempleo que hay en esas provincias. Asimismo, vemos un aporte importante de la actividad principal y de fenómenos de desempleo, como el desempleo agrícola, el cual suele estar vinculado a tener un peso más potente en la decisión del voto en la izquierda.

Finalmente, terminamos el análisis haciendo una interpretación de los coeficientes del modelo:

- **Variable continua (DifComAutonPtge):** el odds ratio de 1.0306 significa que la ODD de que gane la izquierda añadiendo un punto porcentual a la variable DifComAutonPtge es de 1.0306 veces mayor que la ODD del evento sin añadir este punto porcentual, *ceteris paribus*.
- **Variable categórica (CCAA_Madrid):** el odds ratio de 0.4695 significa que la ODD de que la izquierda gane si el pueblo no está en Madrid es 2.1299 veces mayor que la ODD si el pueblo estuviese en Madrid, *ceteris paribus*. Esto tiene sentido ya que históricamente, Madrid es una Comunidad Autónoma que tiende más a la derecha.

5 Conclusión

Observación: Se ha intentado incluir la mayor cantidad de código posible en la entrega, siguiendo la guía que se subió como indicación. Sin embargo, debido a las restricciones en el espacio, hemos optado por excluir algunos de las partes del código que consideramos que no aportan a la entrega (como por ejemplo aquellas dedicadas a la generación de las imágenes y gráficas).

El presente trabajo ha abordado el análisis del comportamiento electoral en los municipios españoles a través de dos modelos de regresión: un modelo de regresión lineal para predecir el porcentaje de abstención (*AbstentionPtge*) y un modelo de regresión logística para predecir si el bloque de izquierdas obtuvo más votos que el de derechas y otros (*Izquierda*).

El proceso comenzó con una exhaustiva fase de depuración del dataset *DatosEleccionesEspaña.xlsx*, compuesto por 8.117 municipios y 41 variables originales. Durante esta fase se identificaron y corrigieron múltiples problemas: valores fuera de rango en variables porcentuales como *Age_over65_pct* y *ForeignersPtge*, códigos centinela en la variable *Explotaciones* (valor 99999), missings no declarados en la variable *Densidad* (categoría '?'), y valores atípicos en variables como *Population* y *TotalCensus*, con porcentajes de outliers del 9,91 % y 9,63 % respectivamente. Todos estos casos fueron tratados mediante su conversión a missing y su posterior imputación aleatoria, garantizando la integridad del conjunto de datos para el análisis posterior.

En cuanto al **modelo de regresión lineal**, el modelo ganador fue el Forward/Stepwise BIC, con 62 parámetros y un R^2 de 0,4302 en train y 0,4239 en test, lo que refleja una capacidad explicativa moderada. Si bien el modelo es robusto, con residuos incorrelados según el estadístico de Durbin-Watson, y generaliza correctamente al conjunto de test sin señales de sobreajuste, no podemos afirmar que se trate de un buen ajuste en sentido estricto, ya que en la práctica se considera que un modelo de regresión lineal es satisfactorio cuando el R^2 supera los valores de 0,6–0,7. Las variables más relevantes fueron *CodigoProvincia*, *SameComAutonPtge* y *ActividadPpal*, lo que pone de manifiesto el fuerte componente territorial y socioeconómico que subyace a la abstención electoral.

Respecto al **modelo de regresión logística**, los resultados fueron notablemente mejores. El modelo ganador, seleccionado mediante Stepwise AIC/BIC, obtuvo un pseudo- R^2 de 0,3314 en train y 0,3479 en test, valores considerados buenos según la literatura, y un área bajo la curva ROC de 0,8787, próxima a 1 e indicativa de una buena capacidad discriminante. El punto de corte óptimo fue determinado mediante el índice de Youden en 0,21, priorizando el equilibrio entre sensibilidad (0,839) y especificidad (0,765), lo que resulta especialmente apropiado en un contexto donde tanto la correcta identificación de municipios de izquierda como la de no izquierda tiene relevancia política. De nuevo, la provincia destacó como la variable más influyente, seguida de *DifComAutonPtge* y *ActividadPpal*.

A pesar de los resultados obtenidos, el trabajo presenta diversas limitaciones que abren la puerta a mejoras y líneas de investigación futuras:

1. Mejora del ajuste mediante transformaciones de variables. En el presente trabajo, el enunciado restringió el uso de transformaciones. Sin embargo, dado que muchas de las variables presentan distribuciones asimétricas, consecuencia natural de la gran heterogeneidad entre municipios rurales y urbanos, la aplicación de transformaciones logarítmicas o de raíz cuadrada sobre variables como *Population*, *TotalCensus* o *Explotaciones* podría mejorar la linealidad de las relaciones y, con ello, el ajuste del modelo de regresión lineal, acercando el R^2 a valores más satisfactorios.

2. Introducción de interacciones entre variables. Una extensión natural de ambos modelos sería la incorporación de términos de interacción entre variables explicativas. Por ejemplo, la interacción entre *CodigoProvincia* y variables de desempleo como *ConstructionUnemploymentPtge* o *AgricultureUnemploymentPtge* podría capturar efectos diferenciales del paro por provincia, que en la actualidad quedan diluidos en los efectos principales. Del mismo modo, la interacción entre *ActividadPpal* y la estructura de edad de la población (*Age_over65_pct*, *Age_19_65_pct*) permitiría modelar cómo el perfil económico del municipio modula el efecto del envejecimiento sobre el voto. Estas interacciones, si bien aumentarían la complejidad del modelo y el riesgo de sobreajuste, podrían mejorar sustancialmente el poder predictivo, especialmente en el modelo lineal.

3. Segmentación del análisis por tamaño municipal. Dado que la alta asimetría de variables como *Population* refleja la enorme brecha entre grandes ciudades y pequeños municipios rurales, podría resultar conveniente construir modelos separados para distintos segmentos de población (por ejemplo, municipios de menos de 1.000 habitantes, de 1.000 a 50.000 y de más de 50.000), obteniendo así modelos más precisos y con mayor capacidad explicativa en cada estrato.

En definitiva, el trabajo ha permitido identificar los principales factores territoriales, demográficos y socioeconómicos asociados tanto a la abstención como al voto de izquierdas en España, obteniendo un modelo logístico de calidad notable y un modelo lineal con margen de mejora. Los resultados son coherentes con la realidad política española y ofrecen una base sólida sobre la que continuar profundizando en el análisis del comportamiento electoral a nivel municipal.